CSE233 Week 13: Recursion

Recursion is a process by which a function calls itself. A function that does so is said to call itself recursively. Not all languages support recursion, but C++ does. Recursion is a powerful tool that can replace loops with simpler, cleaner code. There are also some programming problems when implementing graphs and trees that are only solvable by recursion. However, recursion can also cause problems if you are not careful.

Basic Example: Summing Numbers

Let's consider a function that sums numbers from n to 1, where n is the number input. If n = 5, then we'd sum 5 + 4 + 3 + 2 + 1 = 15.

```
int sumDown(int n) {
    int sum = 0;
    for (int i = n; i > 0; --i) {
        sum += n;
    }

    return sum;
}
```

We can also define this function recursively. Whenever we are defining a recursive function, we need to have a base case. A base case is an input to the function that causes the recursion to stop. In this case, if n == 1, the function will always return 1. If n is less than 1, the function returns 0. These two comprise the base cases of the function. The recursive step is the step that causes the function to call itself. Note that `addDown(2)` is equal to `2 + addDown(1)` which is equal to `2 + 1`. Similiarly, `addDown(3) == 3 + addDown(2) == 3 + 2 + addDown(1) == 3 + 2 + 1`. Therefore, the recursive step for addDown is `n + addDown(n - 1)`. Now that we've specified the base and recursive steps, we can define the recursive function:

```
int addDown(int n) {
    // base step for n = 0 or n being negative
    if (n <= 0) return 0;
    // base step for n = 1
```

```
    else if (n == 1) return 1;
    // recursive step
    else return n + addDown(n - 1);
}
```

More Complex Example: Greatest Common Divisor

The greatest common divisor (GCD) problem is the following: given two integers A and B, find the largest possible integer C that divides both A and B. The Euclidean algorithm is the accepted solution to find a GCD. The Euclidean algorithm successively solves the equation:

$r_{k-2} = q_k r_{k-1} + r_k$

Where r is the remainder, q is the quotient, and k is the number of steps the algorithm has been performed. Two perform the algorithm, we start at step k=0 and replace r_{0-2} with A and r_{0-1} with B. As an example, let's compute the GCD of 42 and 12. We start by plugging in 42 and 12:

Step 0: $42 = q_0 12 + r_0$

Now, we solve the equation for q_0 and r_0:

$42 = 3 * 12 + 6$

Next, we plug in 12 and 6 for the next step of the algorithm:

Step 1: $12 = q_1 6 + r_1$

Solving for q_1 and r_1 yields:

$12 = 2 * 6 + 0$

Once r_k reaches 0, the algorithm is done. The value of r_k from the previous step is the GCD. Therefore, GCD(42, 12) = 6. Let's consider another example:

GCD(99, 45)

Step 0: $99 = q_0 15 + r_0$ => $99 = 2 * 45 + 9$

Step 1: $45 = q_1 9 + r_0$ => $45 = 5 * 9 + 0$

GCD(99, 45) = 9

We can implement a function in C++ to find the GCD of two numbers using the a loop or recursion. First, let's look at the loop implementation:

```
int computeGCD(int a, int b) {
    // swap a and b if a is less than b
    if (a < b) {
        int temp = a;
        a = b;
        b = temp;
    }
    // q is integer division of a and b
    int q = a / b;
    // r is remainder (modulus) of a and b
    int r = a % b;
```

```
    a = b;
    b = r;

    while (r != 0) {
        q = a / b;
        r = a % b;
        a = b;
        b = r;
    }

    // at this point, a is the GCD
    return a;
}
```

We can also implement the GCD using recusion. Note that each time we compute a step of the Euclidean algorithm, we are computing the GCD of B and A MOD B. Therefore, the GCD can be computed recursively until A MOD B is 0.

```
int recursivelyComputeGCD(int a, int b) {
    // swap a and b if a is less than b
    if (a < b) {
        int temp = a;
        a = b;
        b = temp;
    }
    if (b == 0) {
        return a;
    }
    else {
        return recursivelyComputeGCD(b, a % b);
    }
}
```

Conclusion

This week, we discussed recursion. Recursion is an important programming technique, but at this point we can only look at it from a more basic standpoint. Recursion is required for many graph and tree algorithms, but that is a topic for a data structures course.
